Apply transfer learning for character recognition using a pre-trained CNN model (VGG16)

To apply transfer learning using a pre-trained VGG16 model for handwritten digit recognition using the MNIST dataset.

Algorithm
1. Import TensorFlow and the VGG16 pre-trained model.
2. Load the MNIST handwritten digit dataset.
3. Normalize and separate the training and testing images.
4. Resize the images from 28 × 28 to 32 × 32 pixels.
5. Convert grayscale images into three-channel RGB images.
6. Load the pre-trained VGG16 model with ImageNet weights.
7. Freeze the pre-trained convolutional layers to preserve learned features.
8. Add a global average pooling layer to extract features.
9. Add a Dense layer for classification.
10. Add a softmax output layer with 10 neurons representing digits 0–9.
11. Compile and train the model using the MNIST dataset.
12. Evaluate the model using test images and display its accuracy.

Program

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
tf.keras.utils.set_random_seed(20)
# Load MNIST
(x_train, y_train), (x_test, y_test) = (
tf.keras.datasets.mnist.load_data()
)
# Use smaller subsets for faster training
x_train = x_train[:12000]
y_train = y_train[:12000]
x_test = x_test[:2000]
y_test = y_test[:2000]

# Resize and convert grayscale to RGB
def prepare(images):
images = tf.cast(images[..., None], tf.float32)
images = tf.image.resize(images, (32, 32))
images = tf.image.grayscale_to_rgb(images)
return preprocess_input(images)
x_train = prepare(x_train)
x_test = prepare(x_test)
# Load pre-trained VGG16
base = VGG16(
weights='imagenet',
include_top=False,
input_shape=(32, 32, 3)
)
# Freeze pre-trained layers
base.trainable = False
# Create transfer learning model
model = models.Sequential([
layers.Input(shape=(32, 32, 3)),
base,
layers.GlobalAveragePooling2D(),
layers.Dense(128, activation='relu'),
layers.Dropout(0.3),
layers.Dense(10, activation='softmax')
])
# Compile
model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
# Train
model.fit(
x_train, y_train,
```

```
epochs=3,
batch_size=64,
validation_split=0.1
)
# Evaluate
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)

print("VGG16 Test Accuracy:", accuracy * 100, "%")
```

Output of Program

```
Downloading VGG16 ImageNet weights...
Epoch 1/3
Epoch 2/3
Epoch 3/3
VGG16 Test Accuracy: Training-dependent value
```

Result

Thus, transfer learning was successfully implemented using the pre-trained VGG16 model. The pre-trained convolutional layers were reused and a new classifier was trained for handwritten digit recognition.